PROJECT ENCLAVE

Build Pack

Feasibility, open-source research, architecture, security and scalability design, project report (Chapters 1 to 3) and the Claude Code build prompt

Team	Kevin Ojwang (lead developer) · Victor Boore (team member)
Client context	Multi-tenant CCTV monitoring and alerting for Kenyan property operators
Version	1.0 · 6 November 2026

Contents

- **Part A.** Verdict and recommended starting point
- **Part B.** Research: open frameworks and what works
- **Part C.** System design: architecture, data, security, scalability
- **Part D.** Project documentation, Chapters 1 to 3 (corrected)
- **Part E.** Claude Code build prompt for the terminal
- **Part F.** Using REA and other Claude Code skills
- **Part G.** Sources and self-study links

Part A. Verdict and recommended starting point

A.1 Is Enclave feasible?

Yes. Every core capability (camera ingest, browser live view, recording, time-range playback, push alerts and tenant isolation) exists today as mature open-source software. The work is in combining them into a secure multi-tenant product, which a two-person team using Claude Code can bring to a pitchable demo in about 7 weeks, and to a working web vertical slice in a single focused day.

Three conditions apply:

1. **Video must leave client sites through an edge gateway.** Cameras sit behind routers, often carrier-grade NAT, so the cloud cannot pull their streams. A small on-site box pulls camera streams on the LAN and pushes them outbound. Port forwarding is not acceptable: it is a common way cameras are compromised.
2. **Record at the edge, not in the cloud.** One 1080p camera at about 2 Mbit/s produces about 650 GB a month. Few Kenyan sites can upload that for ten cameras. Full-quality recording stays on site; the cloud receives sub-streams, event clips and on-demand live view.
3. **Pitch honestly against existing multi-tenant products.** Eagle Eye Networks and Hik-ProConnect already serve many customers from one platform. Enclave's edge is vendor neutrality, Kenyan hosting and pricing, and built-in data protection support.

A.2 Your idea: a web MVP today, then demo and hardware

The direction is right; the scope for "today" needs to be a vertical slice, not everything.

Going web-first is the best call in this pack. A progressive web app (PWA) runs on desktop and phones from one codebase, and the Web Push standard delivers alerts to desktop browsers, Android, and iPhones (iOS 16.4 and later, once the app is added to the home screen). That removes Flutter and Tauri from the critical path entirely.

What one focused day with Claude Code can realistically produce:

- Docker Compose running PostgreSQL, Redis, MediaMTX and four simulated cameras.
- A FastAPI backend with login, two tenants, camera CRUD and row-level security.
- A React PWA showing a live 2×2 grid over WebRTC, with stream access checked per tenant.
- Recording switched on, with a basic playback page using MediaMTX's playback server.

What will not be solid in one day: motion alerts, push notifications, clip export, audit logs, roles, the edge gateway, security hardening and tests. Plan for **day 1 = vertical slice, weeks 1 to 5 = full web MVP, week 6 = real hardware, week 7 = demo polish** (see the schedule in Section 3.11).

Leaving hardware for last is fine, with one exception: buy one cheap ONVIF camera with pan-tilt (and ideally a Raspberry Pi 5) early. Simulated video cannot test PTZ, discovery or camera motion events, and those are where vendor quirks show up.

A.3 Recommended starting point

Decision	Choice	Why
Client	React + TypeScript PWA	One codebase for desktop and mobile; Web Push for alerts
Media	MediaMTX (MIT)	Ingest, WebRTC/HLS, recording and playback API in one binary; per-path HTTP auth
Backend	FastAPI (Python)	Fast to build, async, OpenAPI docs; Python suits later AI work
Data	PostgreSQL with row-level security; Redis	Database-enforced tenant isolation; event queue and pub/sub
Auth (MVP)	Built-in JWT + Argon2 passwords + TOTP later	Fewer moving parts for the demo
Auth (production)	Keycloak Organizations (Apache 2.0)	Multi-tenant SSO and MFA without AGPL obligations
Edge gateway	MediaMTX on Raspberry Pi 5 or mini-PC; Frigate (MIT) as Phase 2 option	Outbound push, local recording, later AI detection
Object storage (later)	SeaweedFS (Apache 2.0)	MinIO's community edition was archived in 2026
Notifications	Web Push (VAPID) + SMTP; Novu (MIT) if channels grow	Free, standard, no app store needed

A.4 Changes from the earlier research

Earlier plan	Now	Reason
Tauri desktop app + Flutter mobile app	One PWA; native apps deferred	Halves front-end work; Tauri WebRTC is unreliable on Linux
Cloud pulls RTSP from cameras	Edge gateway pushes outbound	Cameras are behind NAT; port forwarding is unsafe
MinIO for recordings	MediaMTX disk recording now, SeaweedFS later	MinIO community edition stopped binaries (Oct 2025) and was archived (2026)
Keycloak from week 1	Built-in JWT for MVP, Keycloak later	Less setup for the demo
FCM/APNs push	Web Push first	No Firebase project or Apple account needed for the demo
8 to 10 weeks solo	About 7 weeks, two people with Claude Code	Narrower client scope and AI-assisted coding
"No one does multi-tenant"	Differentiate on neutrality, local hosting, price	Claim was false
ODPC registration "at scale"	Mandatory for CCTV regardless of size	2021 Registration Regulations

Earlier plan	Now	Reason
GDPR “30-day default”	Retention configurable per tenant	GDPR sets no fixed period

A.5 Legal and market points to carry into the pitch

- **ODPC registration.** CCTV for crime prevention requires registration regardless of turnover. Each client registers as a data controller; the Enclave operator registers as a data processor. Certificates last 24 months.
- **Contracts.** Each tenant signs a data processing agreement with the Enclave operator.
- **DPIA.** Systematic monitoring is high-risk processing (KDPA section 31). Ship a DPIA template with onboarding.
- **Breach notification.** 72 hours to the Data Commissioner (section 43). Audit logs and login alerts support this.
- **Short-stay rentals.** Airbnb has banned indoor cameras in all listings worldwide since March 2024. Pitch gates, parking, corridors and entrances.
- **Selling abroad.** Hikvision and Dahua face US authorisation bans (FCC, 2022) and government procurement restrictions elsewhere. Vendor neutrality is a selling point.
- **Legal review.** This pack is not legal advice. Engage a Kenyan data protection practitioner before any pilot with real footage.

Part B. Research: open frameworks and what works

B.1 How working products are built

Every successful remote monitoring product we reviewed follows the same shape, and Enclave should copy it rather than invent a new one:

1. **A device on site that connects outbound.** Cloud VMS products use a bridge or gateway box on the customer's network; manufacturer clouds use firmware that dials home. ONVIF's draft Profile V standardises the same idea (outbound connection, WebRTC streaming).
2. **Recording close to the camera.** Continuous recording stays on site (camera SD card, NVR or gateway disk). The cloud holds event clips and short buffers.
3. **WebRTC for live view, HLS or MP4 for playback.** WebRTC gives sub-second latency; recorded video is served as fragmented MP4 or HLS so the browser's video element can seek, pause and change speed.
4. **Camera-side motion events where available.** Running motion detection on the server for every camera is expensive. Cameras supporting ONVIF Profile T publish motion events; server-side detection is the fallback.
5. **A thin control plane in the cloud.** Tenants, users, permissions, device registry, alerts and audit live in a conventional web backend. The heavy media path is kept separate so it can scale on its own.

B.2 Open-source building blocks

Need	Project	Licence	What it gives Enclave	Decision
Media server	MediaMTX	MIT	RTSP/SRT/RTMP/WebRTC ingest, WebRTC + HLS out, fMP4 recording, playback API (`/list`, `/get`), per-path auth via HTTP or JWT	**Adopt** (cloud and edge)
Stream adapter	go2rtc	MIT	Many camera sources (RTSP, ONVIF, Tapo, Hikvision ISAPI, DVRIP), WebRTC/MSE/HLS out, two-way audio	**Adopt at edge** when a camera needs a non-RTSP protocol or two-way audio
Edge NVR + AI	Frigate	MIT (name and logo trademarked)	Recording with retention, motion and object detection, MQTT events, WebRTC/MSE live view	**Phase 2 edge option**; Enclave agent bridges Frigate events to the cloud

Need	Project	Licence	What it gives Enclave	Decision
Classic VMS	ZoneMinder	GPL-2.0	Full single-site VMS	**Reference only**; GPL and dated architecture
Classic VMS	Shinobi	Custom dual licence	Single-site VMS	**Avoid**; licence terms unclear for SaaS
Video processing	FFmpeg	LGPL/GPL (build-dependent)	Simulated cameras, snapshots, clip export	**Adopt** as an external binary
Motion detection	OpenCV	Apache 2.0	Frame differencing on sub-streams	**Adopt** for MVP
ONVIF client	onvif-zeep-async (Python)	MIT	Discovery, profiles, PTZ, events	**Adopt** on the edge agent
Database	PostgreSQL	PostgreSQL licence	Row-level security for tenant isolation	**Adopt**
Queue / pub-sub	Redis (or Valkey)	Redis: source-available / AGPL options; Valkey: BSD	Streams for events, pub/sub to WebSockets	**Adopt**; Valkey if licence matters
Identity	Keycloak	Apache 2.0	Organizations (multi-tenancy), SSO, MFA; MediaMTX has a Keycloak JWT tutorial	**Adopt in production**
Identity	ZITADEL	AGPL-3.0	Native multi-tenant orgs	**Avoid for SaaS** unless AGPL obligations are accepted
Notifications	Novu	MIT core	In-app inbox, email, SMS, push, chat with workflows	**Optional** once channels grow beyond push + email
Object storage	MinIO	AGPL; community repo archived 2026	—	**Do not use**
Object storage	SeaweedFS	Apache 2.0	S3-compatible storage for event clips	**Adopt later**
Object storage	Garage	AGPL-3.0	Lightweight geo-distributed S3	Alternative if AGPL is acceptable
Web player	hls.js; MediaMTX WHEP reader	Apache 2.0; MIT	HLS fallback and WebRTC playback in the browser	**Adopt**
Load testing	k6	AGPL-3.0 (tool only, not shipped)	API and viewer load tests	**Adopt**

Licence rule for Enclave. Prefer MIT, Apache 2.0, BSD and PostgreSQL licences for anything shipped or hosted as part of the product. AGPL components are acceptable only as unmodified, separately run services, and only after a deliberate decision, because offering a modified AGPL program as a network service obliges you to publish your changes.

B.3 Key facts verified

- **MediaMTX recording.** Configured per path with `record: yes`, `recordPath`, `recordFormat: fmp4`, `recordSegmentDuration` and `recordDeleteAfter`.
- **MediaMTX playback.** Enabled with `playback: yes` (example address `:9996`). `GET /list?path=&start=&end=` returns recorded spans; `GET /get?path=&start=&duration=&format=mp4` returns a playable clip for any time range. This covers DVR seek and clip export.
- **MediaMTX authentication.** `authMethod: http` posts `user`, `password`, `token`, `ip`, `action`, `path` and `protocol` to your backend for every read or publish; a 2xx response allows it. This is how Enclave enforces tenant isolation on video, not just on the API. `authMethod: jwt` with a JWKS URL is the alternative.
- **ONVIF Profile S** stops accepting new conformance after 31 March 2027; **Profile T** replaces it. **Profile V** release candidate was published 9 July 2026, with final release expected by the end of 2026.
- **MinIO** community edition removed most of its web console in May 2025, stopped publishing binaries and images in October 2025, entered maintenance mode in December 2025 and was archived in 2026.
- **Kenya registration.** The 2021 Registration Regulations require registration regardless of turnover for crime prevention, “including operating security CCTV systems”.

B.4 Gaps that still need hands-on proof

- **Frame-by-frame stepping** in the browser has no native API; it is approximated by seeking one frame interval while paused.
- **iOS Web Push** works only for PWAs installed to the home screen; test on a real iPhone early.
- **Camera interoperability** varies even among ONVIF-conformant models. Test the client’s shortlisted models before signing the vendor contract.
- **Edge gateway resilience** (power cuts, reconnection, local buffering) needs testing on real hardware over a real Kenyan connection.

Part C. System design

This part follows the design stages in order: requirements, architecture, data, interfaces, security, scalability, quality and operations. It is the specification Claude Code builds from (Part E).

C.1 Requirements

Functional requirements (MVP)

ID	Requirement	Priority
FR1	Tenant admins invite users and assign roles: tenant_admin, operator, viewer	Must
FR2	Platform admin creates tenants; tenants are fully isolated	Must
FR3	Tenants organise cameras into sites	Must
FR4	Add, edit, disable cameras (RTSP URL or via edge gateway), with main and sub-stream	Must
FR5	Live view: single camera, 2×2 and 3×3 grids; sub-stream in grids and on mobile, main-stream when maximised	Must
FR6	Camera health: online/offline, last seen, bitrate	Must
FR7	Continuous recording with per-tenant retention	Must
FR8	Playback: timeline of recorded spans, seek, pause, 0.5x to 4x speed, jump to event	Must
FR9	Snapshot and clip export (MP4)	Should
FR10	Motion detection with per-camera zones, sensitivity and schedules	Must
FR11	Alerts: in-app inbox (real-time), Web Push, email; acknowledge with note	Must
FR12	Audit log of logins, live views, playback, exports, config changes	Must
FR13	PTZ (pan, tilt, zoom, presets) for ONVIF cameras	Should
FR14	Edge gateway registration with a one-time enrolment token	Should

ID	Requirement	Priority
FR15	Demo mode: seeded tenants with simulated cameras	Must

Non-functional requirements

ID	Requirement	Target
NFR1	Live latency (WebRTC, LAN)	< 1 s
NFR2	Playback start after seek	< 2 s
NFR3	Alert delay (motion to push)	< 10 s
NFR4	Tenant isolation	Zero cross-tenant access in automated tests (API and video)
NFR5	Security baseline	OWASP ASVS 4.0.3 Level 1; no high/critical findings open
NFR6	Availability (pilot)	99.5 % monthly for the control plane
NFR7	Horizontal scale	API and workers stateless; media nodes added without code change
NFR8	Responsiveness	Usable at 360 px width; Lighthouse PWA installable
NFR9	Data location	Configurable deployment region; Kenya hosting supported
NFR10	Observability	Structured logs, metrics and traces for every service

C.2 Architecture

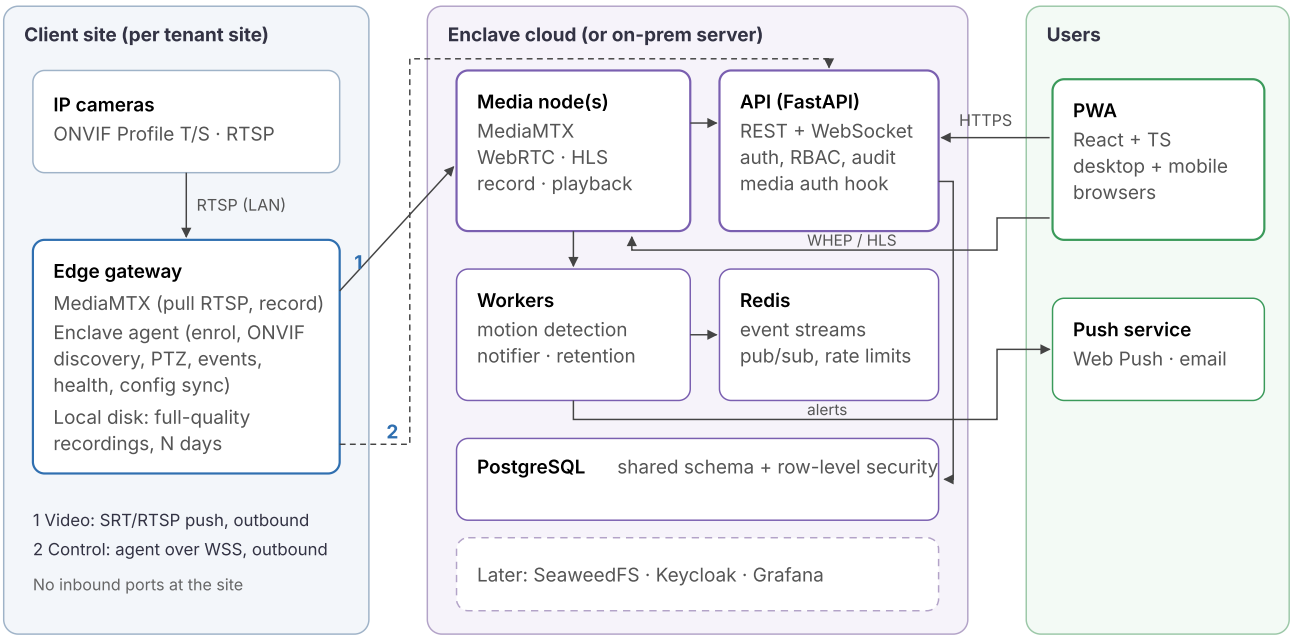


Figure C.1: Enclave architecture. All connections from a client site are outbound, so cameras never need a public address.

Containers and responsibilities

- **Edge gateway.** MediaMTX pulls each camera's RTSP main and sub-stream on the LAN, records the main stream locally and pushes the sub-stream (and main-stream on demand) to a cloud media node over SRT or RTSP. The Enclave agent enrols with a one-time token, then keeps an outbound WebSocket to the API for configuration, ONVIF discovery results, PTZ commands, camera events and health.
- **Media nodes.** MediaMTX instances serving WebRTC (WHEP) and HLS to browsers and serving the playback API. Every read or publish is authorised by calling the API's media auth hook.
- **API.** FastAPI providing REST for all resources, a WebSocket for real-time alerts and camera status, the media auth hook, and the agent channel. Stateless; sessions are JWTs.
- **Workers.** Motion detection (reads sub-streams, publishes events), notifier (fans events out to inbox, Web Push and email with deduplication and quiet hours), retention (deletes expired recordings and metadata).
- **PostgreSQL.** System of record with row-level security. **Redis.** Event streams (consumer groups for workers), pub/sub to API WebSocket nodes, rate limiting.

Key flow: opening a live stream. The PWA asks the API for a stream; the API checks the user's role and camera permission, writes an audit record and returns a WHEP URL plus a short-lived (60 s) stream token. The browser connects to the media node; MediaMTX calls the auth hook with the path and token; the API verifies the token is bound to that tenant, camera and action, and returns 200 or 403.

Key flow: motion alert. The motion worker (or the agent, relaying an ONVIF event) writes an event to a Redis stream. The notifier stores the alert, attaches a snapshot, publishes it to the tenant's WebSocket channel and sends Web Push and email to subscribed users according to their preferences. Acknowledgement is recorded in the audit log.

C.3 Data model

All tenant-owned tables carry `tenant_id` and are protected by row-level security policies of the form `tenant_id = current_setting('app.tenant_id')::uuid`. The API sets `app.tenant_id` per transaction from the authenticated user; the application connects as a role without `BYPASSRLS`.

Table	Key columns
tenants	id, name, slug, region, retention_days, status, created_at
users	id, email (unique), password_hash (Argon2id), totp_secret, is_platform_admin
memberships	user_id, tenant_id, role (tenant_admin, operator, viewer)
sites	id, tenant_id, name, address, timezone
gateways	id, tenant_id, site_id, name, enrol_token_hash, last_seen, version
cameras	id, tenant_id, site_id, gateway_id, name, source_type, main_url_enc, sub_url_enc, onvif_profile, ptz_capable, status, last_seen
camera_permissions	tenant_id, camera_id, user_id, can_view, can_playback, can_export, can_ptz
recordings	id, tenant_id, camera_id, start_at, end_at, location (edge or cloud), bytes
events	id, tenant_id, camera_id, type, started_at, ended_at, score, snapshot_key
alerts	id, tenant_id, event_id, severity, status, acknowledged_by, acknowledged_at, note
alert_rules	id, tenant_id, camera_id, zones (JSON), sensitivity, schedule (JSON), channels
push_subscriptions	id, tenant_id, user_id, endpoint, p256dh, auth
audit_log	id, tenant_id, user_id, action, target_type, target_id, ip, user_agent, at (append-only)

Camera credentials are encrypted at the application level (envelope encryption with a key from the secret store), never returned by the API, and never logged. Media paths follow `t/{tenant_id}/c/{camera_id}/{main|sub}`, so the auth hook can parse tenant and camera from the path.

C.4 Interfaces

- REST under `/api/v1`, documented by OpenAPI. Resources: auth, tenants, users, memberships, sites, gateways, cameras, streams, recordings, events, alerts, alert-rules, push-subscriptions, audit.
- `POST /api/v1/cameras/{id}/stream` returns `{whep_url, hls_url, token, expires_at}`.

- `GET /api/v1/cameras/{id}/recordings?start&end` proxies MediaMTX `/list`; `GET .../clip?start&duration` proxies `/get` with an audit record.
- `POST /internal/media/auth` is the MediaMTX hook (reachable only from media nodes on the private network).
- `WS /api/v1/ws` streams alerts and camera status for the user's tenant.
- `WS /agent/v1` is the gateway channel (mutual authentication with a per-gateway credential issued at enrolment).
- Cursor pagination, idempotency keys on POST, consistent error format (RFC 9457 problem details).

C.5 Security design

Threat model (STRIDE) and controls

Threat	Example	Controls
Spoofing	Stolen password; forged gateway	Argon2id, rate-limited login, TOTP for admins, per-gateway credentials, enrolment tokens single-use and expiring
Tampering	Altered recordings or audit log	Append-only audit table (no UPDATE/DELETE grants), SHA-256 hashes of exported clips recorded in audit, TLS everywhere
Repudiation	"I never exported that clip"	Audit every live view, playback, export, PTZ and config change with user, IP and time
Information disclosure	Tenant A sees tenant B's camera	RLS in Postgres, tenant-bound stream tokens checked by the media auth hook, IDOR tests for every endpoint, encrypted camera credentials
Denial of service	Viewer floods media node	Per-user and per-tenant rate limits and concurrent stream caps, reverse proxy limits, sub-stream defaults
Elevation of privilege	Viewer changes rules	RBAC checks in a single dependency layer, deny by default, tests per role and endpoint

Baseline controls

- TLS 1.2+ at the edge (Caddy or Traefik with automatic certificates); HSTS; secure, HttpOnly, SameSite cookies for refresh tokens; short-lived access tokens (15 min).
- Strict Content Security Policy, no inline scripts; CORS limited to the app origin.
- Input validation with Pydantic; parameterised SQL only (SQLAlchemy); output encoding in React.
- Secrets from environment or a secret manager, never in the repository; `.env.example` only.

- Containers run as non-root with read-only filesystems where possible; images pinned by digest; Trivy scans in CI.
- Dependency scanning (pip-audit, npm audit, Dependabot); secret scanning; Semgrep or Bandit for static analysis.
- Cameras and gateways: no inbound ports at client sites; camera default passwords changed during onboarding; cameras on a separate VLAN where the site allows.
- Privacy: retention enforcement, export watermarking with user and time, data subject request procedure, breach response runbook (72 hours).

C.6 Scalability design

The control plane and the media plane scale independently.

Stage	Size	Deployment
Demo	1 to 10 cameras	Docker Compose on a laptop or one VPS
Pilot	Up to about 100 cameras, a few tenants	One 8 vCPU VPS in Nairobi or a nearby region; Postgres backups; edge recording
Growth	Hundreds of cameras	API and workers behind a load balancer (2+ replicas); 2+ media nodes; managed Postgres with read replica; Redis with persistence
Scale	Thousands of cameras	Kubernetes (K3s or managed); media nodes assigned per site by a placement table; workers autoscaled on Redis stream lag; large tenants optionally on dedicated databases or media nodes

Design rules that make this possible from day one:

- **Stateless API and workers.** No in-memory session state; WebSocket fan-out goes through Redis pub/sub so any API node can serve any user.
- **Media placement.** Each camera has a `media_node` assignment; the API returns that node's URLs. Adding a node means adding a row, not changing code.
- **Bandwidth discipline.** Grids and mobile use sub-streams; main-stream only when a tile is maximised; live streams stop when nobody is watching (MediaMTX on-demand).
- **Edge recording first.** Cloud stores event clips and short buffers only, so cloud storage grows with events, not hours.
- **Queue-based work.** Motion analysis and notifications consume Redis streams with consumer groups, so throughput scales by adding workers.
- **Partitioned tables.** `events`, `alerts` and `audit_log` partitioned by month; indexes lead with `tenant_id`.

- **Observability.** Prometheus metrics (MediaMTX exposes its own), Grafana dashboards, structured JSON logs with request IDs, OpenTelemetry traces.

C.7 Quality, delivery and operations

- **Repository layout (monorepo):** `api/`, `web/`, `workers/`, `edge/`, `infra/` (compose, MediaMTX configs, Caddy), `sim/` (FFmpeg camera simulator), `docs/` (ADRs, diagrams), `tests/e2e/`.
- **Testing:** Pytest unit and API tests (including a cross-tenant isolation suite that runs against every endpoint), Playwright end-to-end tests for login, live grid, playback and alert acknowledgement, k6 load tests for API and concurrent viewers.
- **CI (GitHub Actions):** lint (ruff, eslint), type check (mypy, tsc), tests, security scans (pip-audit, npm audit, Trivy, Semgrep), build images.
- **Architecture decisions** recorded as short ADRs in `docs/adr/`.
- **Operations:** nightly Postgres backups with restore test, uptime checks, alerting on camera offline and media node health, documented runbooks.

Part D. Project Documentation (Chapters 1 to 3, corrected)

This part is the project report as it stands, with every correction from the feasibility review applied. Chapters 4 to 6 follow once the system is designed and built.

Document control

Field	Value
Project name	Enclave
Project type	Multi-tenant remote CCTV monitoring and alerting platform
Document type	Project documentation
Team	Kevin Ojwang (lead developer), Victor Boore (team member)
Version	1.1 (draft)
Status	In progress

Version	Date	Author	Description
0.1	2026	K. Ojwang	Feature brainstorm and scope lock
1.0	2026	K. Ojwang	Chapters 1 and 2 drafted
1.1	6 Nov 2026	K. Ojwang, V. Boore	Feasibility corrections applied; Chapter 3 drafted; team updated

Abstract

Physical security remains a persistent challenge for residential, commercial and hospitality operators in Kenya. Property managers, short-stay accommodation operators, offices and gated communities increasingly rely on CCTV, yet the value of these systems is undermined by fragmented vendor ecosystems, limited remote access, delayed incident awareness and the absence of centralised, multi-site oversight. Enterprise video management software is costly and complex, cloud platforms tie customers to particular camera brands or foreign hosting, and vendor mobile applications do not aggregate across brands or properties.

This project introduces Enclave, a multi-tenant, vendor-neutral remote monitoring and alerting platform. Enclave separates the monitoring application from the camera hardware by adopting open standards (ONVIF, RTSP and WebRTC) and by placing a small edge gateway at each site that connects outbound to the platform, so that cameras need not be exposed to the internet. The platform provides live multi-camera viewing, DVR-style playback, motion-driven notifications, role-

based multi-tenant access and a unified dashboard across sites, delivered first as a progressive web application usable on desktop and mobile.

The system is built from open-source components and can be self-hosted or deployed to the cloud. It is designed for the Kenyan market and the obligations of the Data Protection Act 2019, while remaining portable to other jurisdictions through configurable retention and hosting location. The deliverable for this phase is a pitch-ready minimum viable product demonstrating live monitoring, playback, notifications and tenant isolation, together with a documented integration path for future camera vendors.

List of abbreviations

Abbreviation	Meaning
API	Application Programming Interface
CCTV	Closed-Circuit Television
DPIA	Data Protection Impact Assessment
DSR	Design Science Research
DVR	Digital Video Recorder
FCM	Firebase Cloud Messaging
GDPR	General Data Protection Regulation (EU)
HLS	HTTP Live Streaming
KDPA	Kenya Data Protection Act 2019
MVP	Minimum Viable Product
NAT	Network Address Translation
NVR	Network Video Recorder
ODPC	Office of the Data Protection Commissioner
ONVIF	Open Network Video Interface Forum
PTZ	Pan-Tilt-Zoom
PWA	Progressive Web Application
RBAC	Role-Based Access Control
RLS	Row-Level Security
RTSP	Real-Time Streaming Protocol
SaaS	Software as a Service
SRT	Secure Reliable Transport
VMS	Video Management System
WebRTC	Web Real-Time Communication
WHEP / WHIP	WebRTC-HTTP Egress / Ingestion Protocol

Chapter 1: Introduction

1.1 Background information

Internet-connected surveillance cameras have become inexpensive and capable, with high-definition capture and, increasingly, on-device analytics. The software that gathers this video and turns it into timely information for decision-makers has not kept pace for small and mid-sized operators, particularly in emerging markets.

In Kenya, remote monitoring is of growing interest to short-stay and serviced apartment operators, commercial offices, and gated communities. Operators who oversee several sites typically face a fragmented situation: each camera brand ships its own mobile application, each site has its own recorder, and there is no single view across properties. Incidents are usually discovered after the fact by reviewing footage manually. *(Market sizing to be supported with client interviews and a published market source before submission.)*

The video management market offers three kinds of product. Enterprise platforms such as Milestone XProtect and Genetec Security Center are powerful but costly and require specialist integrators. Cloud platforms such as Eagle Eye Networks and manufacturer services such as Hik-ProConnect do serve many customers from one platform, but they are typically priced in foreign currency, hosted outside Kenya, or work best with one manufacturer's hardware. Free vendor applications such as Dahua DMSS work only with their own brand.

Enclave is conceived to fill the gap between these: an affordable, vendor-neutral, multi-tenant platform that a Kenyan operator can adopt without committing to a camera brand, with data that can be hosted locally. It builds on open standards (ONVIF for discovery and control, RTSP for ingest, WebRTC and HLS for delivery) and uses an on-site edge gateway so that cameras behind ordinary internet connections can be reached securely. This design follows directly from the client's situation: no cameras have been procured, so the platform must work with whichever compliant vendor is chosen.

Video of identifiable people is personal data under the Kenya Data Protection Act 2019. Operating CCTV for security purposes requires registration with the Office of the Data Protection Commissioner regardless of an organisation's size. Enclave is therefore designed with privacy by design from the outset: tenant isolation, audit logging, configurable retention and local hosting options.

1.2 Problem statement

Operators of multi-site property portfolios in Kenya lack an affordable, unified and vendor-neutral means of remotely monitoring their CCTV and receiving timely alerts. The consequences are threefold:

- **Operational blindness.** Managers cannot see across sites in real time; incidents are discovered after the fact by manual review.
- **Vendor fragmentation.** Each brand has its own application and login. A mixed portfolio has no central audit trail or unified alert stream.

- **Delayed awareness.** Without notifications, intrusions and suspicious activity are noticed only if someone happens to be watching a screen.

Existing options are either enterprise systems that are costly and complex, cloud services that are priced and hosted for other markets or tied to a manufacturer, or single-brand consumer applications. There is an underserved middle: a standards-based, multi-tenant platform with live view, DVR-style playback and notifications, priced and hosted for Kenyan small and mid-sized operators.

1.3 Objectives

Main objective. To design, develop and evaluate Enclave, a multi-tenant, vendor-neutral remote CCTV monitoring and alerting platform that enables property operators to view live feeds, review recorded footage and receive security notifications from desktop and mobile devices.

Specific objectives

i. To investigate the limitations of current remote CCTV monitoring options for multi-site operators in Kenya and evaluate open standards (ONVIF, RTSP, WebRTC) as a vendor-neutral integration strategy.

ii. To design a multi-tenant architecture, including an on-site edge gateway, that isolates tenant data, enforces role-based access control and supports per-tenant retention and hosting settings.

iii. To implement a live monitoring subsystem with multi-camera grids, sub-stream and main-stream selection, PTZ control where supported, and connection health monitoring.

iv. To implement a playback subsystem with DVR-style controls: timeline scrubbing, pause, variable speed, snapshot and clip export.

v. To implement an event-driven notification subsystem that detects motion and delivers alerts by browser or mobile push, in-app inbox and email.

vi. To test the platform against functional, performance, security and tenant-isolation criteria, and to document a vendor integration path for future camera hardware.

1.4 Research questions

i. What are the limitations of current remote CCTV monitoring options for multi-site property operators in Kenya?

ii. How effectively can ONVIF, RTSP and WebRTC, combined with an edge gateway, achieve vendor-neutral camera integration in a multi-tenant platform?

iii. What architecture and data model best suit a multi-tenant, privacy-compliant monitoring platform that can be self-hosted or cloud-hosted?

iv. How can live monitoring, playback and notifications be combined into a single user experience on desktop and mobile?

v. How well does the MVP perform against its functional requirements, latency targets, notification reliability and tenant-isolation guarantees?

1.5 Justification

- **Market need.** Multi-site operators in Kenya are poorly served by enterprise pricing, foreign-hosted cloud services and single-brand applications.
- **Vendor neutrality.** The client has not chosen cameras. Building on ONVIF and RTSP keeps procurement open and preserves the client's negotiating position with vendors.
- **Cost and data control.** An open-source stack keeps licence costs low and allows hosting in Kenya for clients concerned about where footage is stored.
- **Team-readiness.** The architecture is modular so that the current two-person team can grow to backend, front-end and operations roles without redesign.

1.6 Scope

The project covers the design, implementation and evaluation of an MVP comprising:

- A multi-tenant backend providing authentication, authorisation, tenant isolation, camera management, recording metadata, alerts and audit logs.
- A media pipeline based on MediaMTX for ingest, WebRTC and HLS delivery, recording and time-range playback.
- An edge gateway prototype that pulls camera streams on the local network and pushes them outbound to the platform.
- A motion detection worker and a notification pipeline delivering browser push, in-app and email alerts.
- A progressive web application providing live view, grids, playback, alerts and administration on desktop and mobile browsers.
- A simulated camera environment, plus one physical ONVIF camera for PTZ and discovery testing.
- A camera vendor requirements specification and technical documentation.

Excluded from this phase: AI analytics (face recognition, licence plate recognition, behaviour analysis), billing, native mobile applications, a desktop application, replacement of on-site recorders, and integration with emergency services.

1.7 Limitations

- **Limited hardware.** Most testing uses simulated streams; only one physical camera is available until the client procures hardware.
- **Small team.** Two people limit the breadth of features in the project period; depth in monitoring, playback and alerts is prioritised.
- **Open-source trade-offs.** Components lack vendor support contracts; compensating engineering effort is needed.
- **Regulatory interpretation.** Compliance is addressed by design, but legal certification requires a data protection practitioner.
- **Network variability.** Upload bandwidth at Kenyan sites varies widely; edge recording and sub-streams reduce but do not remove this constraint.

- **Limited user study.** Usability is evaluated with a small group rather than a field trial.

1.8 Delimitations

- Enclave is a monitoring and alerting platform. It complements on-site recorders and records at the edge gateway where none exist.
- It targets private security operations, not law enforcement or public surveillance.
- Integration is limited to standards-compliant devices (ONVIF Profile T or S, RTSP). Proprietary SDKs are deferred and would be added as adapters.
- Cameras inside private spaces are out of scope; for short-stay listings, only exterior and common areas are supported, in line with platform policies such as Airbnb's ban on indoor cameras.
- The interface is English-only, with internationalisation scaffolding in place.
- Evaluation uses functional, performance and security criteria; long-term outcomes such as crime reduction are out of scope.

Chapter 2: Literature Review

2.1 Introduction

This chapter reviews the technologies, products and research relevant to Enclave. It covers the video management landscape, the standards that make vendor-neutral integration possible, multi-tenant architecture, notification systems, related works, the gaps Enclave addresses, and a conceptual framework. Sources are a mix of academic literature, standards documents and industry documentation.

2.2 Video management systems

2.2.1 Enterprise platforms

Milestone XProtect and Genetec Security Center lead the enterprise market. XProtect is offered in tiers from small installations to centrally managed multi-site deployments, with video walls, failover recording and incident management. Genetec Security Center unifies video, access control and other systems, and its Clearance module supports secure evidence sharing with audit trails. Both are designed around one organisation per installation, are licensed per camera at enterprise prices, and are usually deployed by certified integrators.

2.2.2 Cloud and manufacturer platforms

Cloud video surveillance (sometimes called VSaaS) platforms such as Eagle Eye Networks host video for many customer organisations on shared infrastructure, typically using an on-site bridge device to connect cameras. Manufacturer services such as Hikvision's Hik-ProConnect let installers manage many customer sites from one portal. These platforms prove that multi-tenant remote monitoring is commercially viable. Their limitations for Enclave's market are cost in foreign currency, hosting outside Kenya, and, for manufacturer services, a strong preference for their own hardware.

2.2.3 Vendor applications

Free applications such as Hikvision iVMS-4200 and Dahua DMSS provide live view, playback and push notifications for their own brand's devices. They are easy to use but cannot combine cameras from different manufacturers or provide a cross-site audit trail.

2.2.4 Open-source systems

Frigate is an open-source NVR with real-time local object detection, recording, a web interface with user accounts, notifications and an MQTT event stream. ZoneMinder is a long-standing open-source VMS supporting many cameras, PTZ and an API. Both demonstrate that vendor-neutral recording and detection are achievable with open software, but each is designed as a single installation for one owner rather than a shared multi-tenant service.

2.3 Standards for vendor-neutral integration

2.3.1 ONVIF

ONVIF, founded in 2008, standardises how IP security products communicate, and is supported by most major manufacturers. Functionality is grouped into profiles:

- **Profile S** covers basic streaming and control. ONVIF will stop accepting new Profile S conformance submissions after 31 March 2027 because its authentication method is now considered weak; existing conformant products remain conformant.
- **Profile T** is the recommended replacement, adding H.265, stronger (digest) authentication, metadata and motion and tampering events.
- **Profile V** is a draft standard for cloud video. Its release candidate (9 July 2026) specifies that devices make a secure outbound connection to a cloud VMS and stream over WebRTC, removing the need for port forwarding. ONVIF expects to finalise it by the end of 2026.

For Enclave, Profile T should be required of new cameras, Profile S accepted for existing ones, and Profile V tracked as a future way to connect cameras without a gateway.

2.3.2 RTSP

The Real-Time Streaming Protocol (RFC 2326; version 2.0 in RFC 7826) controls delivery of camera streams and is supported by virtually all IP cameras. It is Enclave's ingest protocol on the local network.

2.3.3 WebRTC, WHIP and WHEP

WebRTC (overview in RFC 8825) delivers audio and video to browsers with latency typically under one second and with mandatory encryption. WHIP (RFC 9725) and the WHEP draft define simple HTTP signalling for publishing and playing WebRTC streams, and are supported by the MediaMTX media server used by Enclave. HLS is used as a fallback for networks where WebRTC is blocked.

2.3.4 Reaching cameras behind NAT

Most cameras sit behind routers that use network address translation, frequently carrier-grade NAT on Kenyan consumer and business lines, so they have no public address. Opening router ports exposes cameras to automated attacks. The established approaches are a VPN, an on-site bridge that connects outbound (used by cloud VMS products), or, in future, ONVIF Profile V. Enclave adopts the outbound edge gateway.

2.4 Multi-tenant architecture

Multi-tenancy lets one deployment serve many customer organisations while keeping their data separate (Chong and Carraro, 2006; Bezemer and Zaidman, 2010). Three database patterns are common: a database per tenant, a schema per tenant, and a shared schema with a tenant identifier on every row. The shared-schema approach is the most economical and the easiest to migrate, at the cost of relying on the application and database to enforce isolation (Krebs, Momm and Kounev, 2012). PostgreSQL row-level security allows that isolation to be enforced by the database itself, so a programming mistake in one query cannot expose another tenant's data. Enclave uses a shared schema with row-level security, with the option to move large tenants to dedicated databases later.

Access within a tenant follows role-based access control, with roles such as administrator, operator and viewer, and per-site or per-camera permissions.

2.5 Notification systems

Firebase Cloud Messaging and Apple Push Notification service deliver notifications to native mobile apps. For web applications, the W3C Push API with VAPID keys delivers notifications to desktop browsers, to Android, and to iOS 16.4 and later when the application is installed to the home screen. This allows a progressive web application to send alerts without a native app. Commercial VMS products combine push with email and in-app inboxes, and support acknowledgement of alerts; Enclave follows the same pattern.

2.6 Related works

2.6.1 Milestone XProtect

XProtect is the benchmark enterprise VMS, with tiered products up to centrally managed multi-site deployments, video walls, failover recording and incident management. It shows what a mature VMS offers. Its single-organisation design, per-camera licensing and integrator-led deployment place it out of reach for the small and mid-sized operators Enclave targets.

2.6.2 Frigate NVR

Frigate is an open-source NVR with local AI object detection, recording, notifications and WebRTC live view, typically run at home with Home Assistant. It validates the RTSP-in, WebRTC-out pipeline Enclave uses and is a candidate component for Enclave's edge gateway in a later phase. It is a single-owner installation with no tenant model or multi-site management, so it complements rather than competes with Enclave.

2.6.3 Dahua DMSS

DMSS is a mobile-first application offering remote monitoring, push alerts with event context, PTZ control and device sharing. Its alerting experience is a useful reference. It works only with Dahua devices and lacks centralised multi-site management.

2.6.4 Summary

The three works represent the enterprise VMS, the open-source NVR and the vendor application. Together with the cloud platforms in Section 2.2.2, they show that each part of Enclave is proven. What is missing for Kenyan small and mid-sized operators is the combination of vendor neutrality, multi-tenancy, local hosting, affordable pricing and built-in data protection support.

2.7 Gaps in related works

1. **Affordable, vendor-neutral multi-tenancy.** Multi-tenant platforms exist, but they are priced for other markets or tied to a manufacturer; open-source systems are vendor-neutral but single-tenant.

2. **Local hosting.** Few platforms offer hosting in Kenya, which matters to clients concerned about where footage of their premises is stored.
3. **Secure remote access without port forwarding** in vendor-neutral, open-source form.
4. **Data protection support.** Few products help operators meet KDPA duties such as audit logs, retention limits, DPIA support and breach response.

2.8 Conceptual framework

Enclave's framework has six layers:

1. **Cameras.** Any ONVIF or RTSP camera on the site network.
2. **Edge gateway.** Discovers cameras, pulls their streams, records locally and pushes streams outbound to the platform.
3. **Media layer.** MediaMTX receives streams, serves WebRTC and HLS to clients, records and serves time-range playback.
4. **Events and notifications.** A motion worker publishes events to a queue; a notifier delivers push, in-app and email alerts.
5. **Application services.** The API for tenants, users, roles, cameras, recordings, alerts and audit, backed by PostgreSQL with row-level security.
6. **Clients.** A progressive web application on desktop and mobile browsers.

2.9 References for Chapter 2

- Bezemer, C.-P. and Zaidman, A. (2010). Multi-tenant SaaS applications: maintenance dream or nightmare? *Proceedings of IWPSE-EVOL*, ACM.
- Chong, F. and Carraro, G. (2006). *Architecture strategies for catching the long tail.* Microsoft.
- Krebs, R., Momm, C. and Kounev, S. (2012). Architectural concerns in multi-tenant SaaS applications. *Proceedings of CLOSER*.
- IETF RFC 2326 (1998) and RFC 7826 (2016), Real-Time Streaming Protocol.
- IETF RFC 8825 (2021), Overview: Real-Time Protocols for Browser-Based Applications.
- IETF RFC 9725 (2025), WebRTC-HTTP Ingestion Protocol (WHIP).
- ONVIF (2025). Profile S Deprecation Q&A. onvif.org.
- ONVIF (2026). ONVIF Releases Profile V Draft for Cloud Video Without Vendor Lock-In. onvif.org.
- W3C. Push API. w3.org/TR/push-api.

Chapter 3: Methodology

3.1 Introduction

This chapter describes how Enclave is researched, built and evaluated: the research design, data sources, development approach, analysis and design techniques, tools, evaluation criteria, ethical and legal safeguards, deliverables and schedule.

3.2 Research design

The project follows Design Science Research (Hevner et al., 2004; Peffers et al., 2007), which suits projects whose main contribution is a purpose-built artefact evaluated against the problem it addresses. Its six activities map onto the project as follows:

1. **Problem identification.** Fragmented, vendor-locked and costly CCTV monitoring for multi-site operators (Chapter 1).
2. **Objectives of a solution.** A multi-tenant, vendor-neutral platform with live view, playback and notifications (Section 1.3).
3. **Design and development.** Iterative construction of the MVP (Sections 3.4 to 3.7; Chapters 4 and 5).
4. **Demonstration.** A scripted client demonstration with simulated cameras, one physical ONVIF camera and two isolated demo tenants.
5. **Evaluation.** Functional, performance, security, isolation and usability testing (Section 3.8).
6. **Communication.** This documentation, the client pitch and the vendor requirements specification.

The research is applied and mixed-method, combining quantitative measures (latency, alert delay, seek time, resource use) with qualitative input from the client and test users.

3.3 Data acquisition

Requirements data comes from semi-structured interviews with the client, review of target customers' workflows, analysis of the KDPA and its 2021 regulations, and the product review in Chapter 2.

Video data comes from three sources:

- **Simulated cameras.** Licensed stock footage of gates, parking areas, corridors and lobbies, looped with FFmpeg and published as RTSP, each with a main stream and a sub-stream.
- **Controlled motion clips** with known motion at known times, providing ground truth for detection accuracy and alert delay.
- **One physical ONVIF camera** with pan and tilt, to validate discovery, PTZ and camera-generated events.

No footage of members of the public is collected. **System data** (timing logs, resource use, audit events) is recorded during testing for the evaluation in Chapter 5.

3.4 Development approach

Development follows an Agile approach adapted from Scrum (Schwaber and Sutherland, 2020) for a two-person team:

- **Roles.** Kevin Ojwang acts as product owner and lead developer; Victor Boore contributes development, testing and documentation; the client reviews each increment.
- **Sprints.** One week, with planning on Monday and review and retrospective on Friday.
- **Backlog.** User stories on a GitHub Projects board, tagged MVP, post-pitch or Phase 2.
- **AI-assisted development.** Claude Code is used as a pair programmer under a written specification (Part E of the build pack). Every change is reviewed by a team member, tested, and security-reviewed before merging.
- **Definition of done.** Merged to main, automated tests passing, deployed to the demo environment, recorded in the changelog.

Figure 3.1: Adapted Agile Development Cycle shows the loop: backlog, sprint planning, one-week sprint, increment, client review, retrospective.

3.5 System analysis

Requirements are written as user stories grouped by actor (platform administrator, tenant administrator, operator, viewer) and prioritised with MoSCoW. Chapter 4 presents functional and non-functional requirements, a use case diagram, a context diagram, a data flow diagram, and sequence diagrams for opening a live stream and delivering a motion alert.

3.6 System design

The design separates the platform into replaceable layers following Section 2.8: edge gateway, media layer, application layer, data layer (PostgreSQL with row-level security, Redis) and client layer (progressive web application). Security is designed in through threat modelling with STRIDE and verification against the OWASP Application Security Verification Standard. The architecture is documented with the C4 model (Brown, 2018). Chapter 4 presents the entity relationship diagram, database schema, architecture diagram and wireframes.

3.7 Tools and techniques

Layer	Tool	Reason
Media server	MediaMTX	RTSP, SRT, WebRTC and HLS in one binary, with recording and playback
Stream simulation	FFmpeg	Publishes looped footage as RTSP; snapshots and clip export
Motion detection	OpenCV (Python)	Frame differencing on sub-streams
Backend	FastAPI (Python)	Async REST and WebSocket, automatic OpenAPI docs

Layer	Tool	Reason
Database	PostgreSQL 16	Row-level security for tenant isolation
Queue and real-time	Redis	Event streams and publish/subscribe
Web client	React, TypeScript, Vite	Responsive PWA for desktop and mobile
Notifications	Web Push (VAPID), SMTP	Browser push on desktop, Android and installed iOS PWAs; email fallback
Edge gateway	MediaMTX on a Raspberry Pi 5 or mini-PC	Same media server at the edge, outbound push
Deployment	Docker Compose	Reproducible local and single-server deployment
Testing	Pytest, Playwright, k6	Unit and API, end-to-end browser, load tests
Project management	Git, GitHub, GitHub Projects	Source control and sprint board
Diagramming	draw.io, PlantUML	UML and architecture diagrams

3.8 Evaluation approach

Area	Method	Target
Functional correctness	Test cases per feature (Tables 5.3 to 5.6)	All MVP cases pass
Live latency	Burned-in clock compared with displayed frame	Under 1 s on LAN over WebRTC
Playback	Time to start after seeking	Under 2 s
Alert delay	Known motion to push notification	Under 10 s
Tenant isolation	Automated cross-tenant access attempts via API and stream URLs	Zero successes
Security	OWASP ASVS Level 1 checklist; dependency and container scanning	No high or critical findings open
Capacity	Load test with increasing simulated cameras and viewers	Camera count at 80 % CPU reported
Usability	Task sessions with 5 users and the System Usability Scale	SUS of 68 or above

3.9 Ethical and legal considerations

- Only licensed or consented footage is used in development.
- Each client tenant is the data controller; the operator of Enclave is a data processor. ODPC registration is required for CCTV regardless of turnover, and a data processing agreement

will be prepared.
- The platform provides tenant isolation, role-based access, audit logs of viewing and export, configurable retention, encryption in transit and support for 72-hour breach notification.
- Onboarding reminds users not to point cameras at private spaces, and that short-stay listings may not use indoor cameras.
- A data protection practitioner will review the platform before any pilot with real footage.

3.10 Deliverables

1. Working MVP (backend, media pipeline, edge gateway prototype, PWA), deployable with Docker Compose.
2. Simulated camera environment started with one command.
3. Camera vendor requirements specification.
4. Test and security reports.
5. Technical documentation: architecture, schema, API reference, deployment guide.
6. This project documentation (Chapters 1 to 6).
7. Client demonstration script and recorded demonstration.

3.11 Project schedule

Week	Sprint goal
1	Environment, backend skeleton, tenant schema with RLS, authentication, simulated cameras, live view in the PWA (vertical slice)
2	Camera management, grids, connection health, edge gateway prototype
3	Recording and playback: timeline, pause, speed, snapshot, clip export
4	Motion detection, alert pipeline, inbox, Web Push and email
5	Roles, audit log, second demo tenant, retention, security hardening
6	Physical ONVIF camera: discovery and PTZ; evaluation and load tests
7	Documentation, demo script, recorded demo, buffer

With AI-assisted development the schedule is shortened from the earlier 10-week plan to about 7 weeks; the buffer week absorbs video-related debugging, which is the most common source of delay.

3.12 References for Chapter 3

- Brown, S. (2018). *The C4 model for visualising software architecture.* c4model.com.
- Hevner, A. R., March, S. T., Park, J. and Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105.
- OWASP Foundation. *Application Security Verification Standard 4.0.3.*

- Peffers, K., Tuunanen, T., Rothenberger, M. A. and Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3), 45–77.
- Republic of Kenya (2019). *The Data Protection Act, No. 24 of 2019.*
- Schwaber, K. and Sutherland, J. (2020). *The Scrum Guide*. scrumguides.org.

Part E. Claude Code build prompt

E.1 How to use it

1. Create an empty GitHub repository (for example `enclave`), clone it and open a terminal in it.
2. Copy this PDF into the repository as `docs/enclave-build-pack.pdf`.
3. Install Docker, Node.js 20+, Python 3.12+, FFmpeg and Claude Code. Start Claude Code with `claude`.
4. Paste the prompt in E.2 as your first message. Work through the milestones one at a time; review and commit after each.
5. After each milestone run `/security-review` and `/code-review`, and read the diff yourself before merging.

E.2 The prompt

```
You are the lead engineer on Enclave, a multi-tenant, vendor-neutral CCTV
monitoring and alerting platform for Kenyan property operators (offices,
gated communities, short-stay operators). The team is Kevin Ojwang (lead)
and Victor Boore. Read docs/enclave-build-pack.pdf first, especially
Part A (decisions), Part B (frameworks) and Part C (system design). Treat
Part C as the specification. If something in it is wrong or unclear, stop
and tell me before deviating.

GOAL
Build a web MVP (React PWA + FastAPI + MediaMTX + PostgreSQL + Redis) that
demos: two isolated tenants, simulated cameras, live grid over WebRTC,
recording and DVR-style playback, motion alerts with in-app, Web Push and
email delivery, roles and an audit log. Everything runs with
`docker compose up` and a seed script. No native mobile or desktop apps.

STACK (do not substitute without asking)
- infra: Docker Compose; Caddy reverse proxy with TLS for local dev
  (internal CA); MediaMTX (pin a recent release) for ingest, WebRTC/WHEP,
  HLS, fMP4 recording (record, recordPath, recordFormat: fmp4,
  recordSegmentDuration, recordDeleteAfter) and the playback server
  (/list and /get). MediaMTX authMethod: http pointing to the API hook.
- sim/: FFmpeg containers publishing 4-6 looped royalty-free clips as RTSP
  main (1080p) and sub (360p) streams, plus a clip with motion at known
  times for tests.
- api/: Python 3.12, FastAPI, SQLAlchemy 2 (async), Alembic, Pydantic v2,
  Argon2id passwords, JWT access tokens (15 min) + rotating refresh tokens
  in HttpOnly SameSite cookies, pywebpush for Web Push, aiosmtplib email.
- workers/: Python; motion worker using OpenCV frame differencing on
  sub-streams with per-camera zones, sensitivity and schedule; notifier
  consuming Redis streams with consumer groups; retention worker.
- web/: React + TypeScript + Vite PWA (service worker, manifest, Web Push
  subscription), TanStack Query, a WHEP player for live view with hls.js
  fallback, playback page with timeline of recorded spans, seek, pause,
  0.5x-4x speed, snapshot and clip export. Responsive from 360 px.
- edge/: (milestone 6) gateway agent in Python using onvif-zeep-async for
  discovery and PTZ; MediaMTX config that pulls LAN RTSP and pushes to the
  cloud over SRT; enrolment with a one-time token; outbound WebSocket.
```

```
NON-NEGOTIABLE SECURITY AND TENANCY RULES
1. Every tenant-owned table has tenant_id and a PostgreSQL row-level
   security policy. The app connects as a role without BYPASSRLS and sets
   app.tenant_id per transaction. Migrations create the policies.
2. Media paths are t/{tenant_id}/c/{camera_id}/{main|sub}. The API issues
   60-second stream tokens bound to user, tenant, camera and action. The
   MediaMTX auth hook (/internal/media/auth, private network only) rejects
   anything else. Publishing is allowed only for simulator and gateway
   credentials.
3. RBAC roles: platform_admin, tenant_admin, operator, viewer, plus
   per-camera permissions. Deny by default, enforced in one dependency
   layer, tested per role and endpoint.
4. Camera URLs and credentials are encrypted at rest, never returned by
   the API, never logged.
5. Audit every login, live view, playback, export, PTZ command, alert
   acknowledgement and configuration change in an append-only table.
6. Rate-limit login and stream requests; cap concurrent streams per user.
7. Secrets only from environment; commit .env.example, never .env.
8. Strict CSP, CORS limited to the app origin, secure headers via Caddy.
9. Containers run as non-root; pin image versions.
10. Write a cross-tenant isolation test suite that, for every endpoint and
    for stream URLs, proves a user of tenant A cannot read or act on
    tenant B's data. It must pass before any milestone is complete.

SCALABILITY RULES
- API and workers are stateless; WebSocket fan-out goes through Redis
  pub/sub. Each camera has a media_node assignment the API uses to build
  URLs. Grids use sub-streams; main-stream only when a tile is maximised.
  Streams are on-demand. events, alerts and audit_log are partitioned by
  month with indexes leading on tenant_id. Expose Prometheus metrics and
  JSON logs with request IDs.

WORKING METHOD
- First, create CLAUDE.md summarising these rules, the repo layout
  (api/, web/, workers/, edge/, infra/, sim/, docs/adr/, tests/e2e/) and
  the commands to run, lint and test. Then propose a plan for milestone 1
  and wait for my go-ahead.
- Work in small, reviewable commits on a feature branch per milestone.
- Write tests with the code: pytest for API and workers, Playwright for
  end-to-end flows. Run lint, type checks and tests before saying a task
  is done, and show me the output.
- Record significant decisions as short ADRs in docs/adr/.
- Use up-to-date library documentation; do not guess APIs. If a library
  behaves differently from this brief, tell me.
- Never disable a test or a security control to make something pass.

MILESTONES
M1 (vertical slice, target: day 1): compose stack, sim cameras, tenants,
   users, login, cameras CRUD, RLS, stream tokens + MediaMTX auth hook,
   live 2x2 grid in the PWA, isolation tests for these endpoints, seed
   script creating two demo tenants.
M2: sites, camera health (online/offline, last seen, bitrate), 3x3 grid,
   maximise to main-stream, roles and per-camera permissions, audit log.
M3: recording on, retention per tenant, playback page (timeline from
   /list, seek, pause, speed, jump-to-event), snapshot, clip export via
   /get with audit and SHA-256 hash.
M4: motion worker, alert rules (zones, sensitivity, schedule), notifier,
   real-time inbox over WebSocket, Web Push (VAPID), email, acknowledge.
```

```
M5: hardening: TOTP for admins, rate limits, CSP, dependency and image
   scanning in GitHub Actions CI, k6 load test script, backups, runbook.
M6: edge gateway agent and config, ONVIF discovery and PTZ against a real
   camera, SRT push to cloud, local recording, enrolment flow.
M7: demo mode polish, demo script in docs/, README with setup steps.

Start now with CLAUDE.md and the M1 plan.
```

E.3 Tips for working with Claude Code on this

- Keep one milestone per session or branch; start each with “Read CLAUDE.md and docs/enclave-build-pack.pdf Part C, then plan M2.”
- When video does not play, ask Claude to check, in order: the MediaMTX logs, the auth hook’s response, browser `chrome://webrtc-internals`, and codec (H.264 is safest for browsers; H.265 support varies).
- Ask for the isolation test suite output at the end of every milestone.
- Victor can run Playwright tests and review pull requests in parallel; split work by folder (`web/` and `api/`) to avoid conflicts.

Part F. Using REA and other Claude Code skills

F.1 What REA is

REA (“Reverse Engineer Anything”) is an MIT-licensed MCP server and skill that lets an AI agent inspect compiled applications, JavaScript and Electron apps, .NET assemblies and websites, using engines such as Ghidra, Hopper or IDA. It is a reverse-engineering toolkit, not a video or web framework.

F.2 Where it helps Enclave, and where it does not

Not needed for the build. Everything Enclave integrates with is documented: ONVIF specifications, RTSP, WebRTC, MediaMTX and Frigate are all open. Reverse engineering adds no value there.

Do not use it on vendor apps such as Hik-Connect, DMSS or iVMS. Their licence terms typically prohibit reverse engineering, and copying their implementation into Enclave risks copyright and contract claims from exactly the vendors the client wants to partner with. Study their features from the outside (screens, documentation, public reviews) instead, which is how Chapter 2 does it.

Legitimate, limited uses:

- Inspecting the web interface of a camera you own to find its RTSP paths or ONVIF quirks, when the manufacturer’s documentation is missing. Prefer asking the vendor for their integration guide or SDK first, and check their terms.
- Auditing Enclave’s own production build (for example, checking that no secrets are bundled into the PWA’s JavaScript).
- Learning: analysing open-source software with permissive licences.

If you install it, keep it out of the Enclave repository’s default setup so that other contributors are not exposed to it unknowingly.

F.3 Skills and tools that do help

Tool	Use in Enclave
`/init` (built in)	Generates `CLAUDE.md`; the prompt asks Claude to write one first
`/security-review` (built in)	Run on every milestone branch before merging
`/code-review` (built in)	Correctness review of each diff
Playwright (pre-installed in many environments, or `npm i -D @playwright/test`)	End-to-end tests for login, live grid, playback and alerts
Frontend design skill (Anthropic plugin marketplace)	Better-looking PWA screens for the pitch
A documentation MCP such as Context7	Current docs for MediaMTX, FastAPI and React, reducing guessed APIs
GitHub CLI (`gh`)	Pull requests and CI status from the terminal

Part G. Sources and self-study links

Grouped by topic. These replace the earlier link list, which pointed to Korean, Chinese and Hebrew versions of pages and to an unofficial DMSS site.

Media and streaming

1. MediaMTX: https://github.com/bluenviron/mediamtx
2. MediaMTX recording: https://mediamtx.org/docs/features/record
3. MediaMTX playback: https://mediamtx.org/docs/features/playback
4. MediaMTX authentication: https://mediamtx.org/docs/features/authentication
5. go2rtc: https://github.com/AlexxIT/go2rtc
6. Frigate NVR: https://github.com/blakeblackshear/frigate
7. ZoneMinder: https://zoneminder.com/
8. FFmpeg: https://ffmpeg.org/
9. hls.js: https://github.com/video-dev/hls.js

Standards

10. ONVIF Profile S deprecation Q&A: https://www.onvif.org/profiles-2/profile-s/profile-s-deprecation-qna/
11. ONVIF Profile V release candidate press release: https://www.onvif.org/?p=9145
12. ONVIF profiles overview: https://www.onvif.org/profiles/
13. RTSP 2.0, RFC 7826: https://www.rfc-editor.org/rfc/rfc7826
14. WebRTC overview, RFC 8825: https://www.rfc-editor.org/rfc/rfc8825
15. WHIP, RFC 9725: https://www.rfc-editor.org/rfc/rfc9725
16. W3C Push API: https://www.w3.org/TR/push-api/

Backend, identity, storage, notifications

17. FastAPI: https://fastapi.tiangolo.com/
18. PostgreSQL row-level security: https://www.postgresql.org/docs/current/ddl-rowsecurity.html
19. Keycloak: https://www.keycloak.org/ (release notes: https://www.keycloak.org/docs/latest/release_notes/index.html)
20. ZITADEL: https://github.com/zitadel/zitadel
21. Novu: https://github.com/novuhq/novu
22. SeaweedFS: https://github.com/seaweedfs/seaweedfs
23. MinIO archived, alternatives: https://pinggy.io/blog/minio_archived_self_hosted_s3_alternatives/
24. MinIO moves away from open source: https://itsfoss.com/news/minio-moves-away-from-open-source/

Security

25. OWASP ASVS: https://owasp.org/www-project-application-security-verification-standard/
26. OWASP Top 10: https://owasp.org/www-project-top-ten/
27. C4 model: https://c4model.com/

Products reviewed

28. Milestone XProtect: https://www.milestonesys.com/products/software/xprotect-comparison/
29. Genetec Security Center: https://www.genetec.com/products/unified-security/security-center
30. Eagle Eye Networks: https://www.een.com/
31. Hik-ProConnect: https://www.hik-proconnect.com/
32. Dahua DMSS (official): https://www.dahuasecurity.com/

Law and policy

33. Kenya Data Protection Act 2019 (Kenya Law): https://kenyalaw.org/kl/fileadmin/pdfdocuments/Acts/2019/TheDataProtectionAct__No24of2019.pdf
34. ODPC: https://www.odpc.go.ke/
35. Registration Regulations 2021 summary (Cliffe Dekker Hofmeyr): https://www.cliffedekkerhofmeyr.com/news/publications/2022/Practice/TMT/tmt-alert-14-february-Part-3-Unpacking-the-Data-Protection-Registration-of-Data-Controllers-and-Data-Processors-Regulations-of-2021.html
36. Airbnb indoor camera ban (AP, March 2024): https://www.wsls.com/business/2024/03/11/airbnb-is-banning-the-use-of-indoor-security-cameras-in-the-platforms-listings-worldwide/

Tools

37. REA: https://github.com/morluto/rea
38. Claude Code documentation: https://code.claude.com/docs

Prepared 6 November 2026. Facts marked as verified in Part B were checked against the linked pages on that date. This pack is not legal advice.